Autodesk Forge is now Autodesk Platform Services
Learn More

Share

← Back to Community Blog

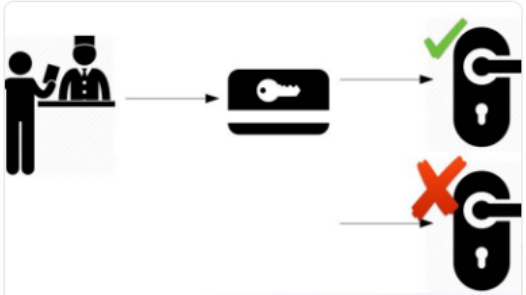
30 MAY 2022

Buckets Tools: migrate Node.js app to Electron

Adam Nagy



Related Article



9 FEB 2023

Scoped Tokens are Here !

INTRODUCTION Tokens are the core of Forge/APS platform's access control mechanism. Tokens act like a temporary password, and grant access to...

Viewer, Authentication, Model Derivative API, Data Management API

Bucket Tools: migrate Node.js app to Electron

Michael Beale

Some Forge users were asking for a way to interact with OSS without having to install Visual Studio Code (in order to use Autodesk Forge Tools extension) or go to a webpage like https://oss-manager.autodesk.io/ (**Buckets Tools**)

So I looked into migrating the existing **Buckets Tools** Node.js app to an Electron app.

Download Mac Version: **HERE**

and window version: **HERE**

Steps to Migrating Node.JS app to Electron

I followed the Electron Quick Start.

First I had to go into the folder of the app and add **Electron** to it:

```
npm i electron –save-dev
```

Then add **BrowserWindow** in the **start.js** file. I renamed the **express** app from **app** to **expressApp** so that I could just reference the **app** object from the **electron** library:

```
var expressApp = require('./server/server');

const { app, BrowserWindow } = require('electron');
app.on('ready', function() {
  let mainWindow = new BrowserWindow({
    width: 1280,
    height: 720,
    webPreferences: {
      nodeIntegration: true
    },
    autoHideMenuBar: true,
    useContentSize: true,
    resizable: true,
  });
  mainWindow.loadURL('http://localhost:3000/');
  mainWindow.focus();
});
```

```
// start server
var server = expressApp.listen(expressApp.get('port'), function () {
  console.log('Starting at ' + (new Date()).toString());
  console.log('Server listening on port ' + server.address().port);
});
```

Since I was using jQuery in the project, I also needed to include it somehow. First I found this solution, but then when tried to include jsTree as well, it did not help.
Then found this solution that helped with the inclusion of all JavaScript libraries in my **index.html** file ?

```
<!-- Insert this line above script imports  -->
<script>if (typeof module === 'object') {window.module = module; module = undefined;} </script

    <script language="JavaScript" src="/js/jquery.min.js"></script>
    <script language="JavaScript" src="/js/jquery.storageapi.min.js"></script>
    <script src="/js/jstree.js"></script>
    <script src="/js/bootstrap.min.js"></script>

<!-- Insert this line after script imports -->
<script>if (window.module) module = window.module;</script>
```

Since I like starting debugging using the **Visual Studio Code**'s debug button, I needed to set things up in the **launch.json** file. I could follow this solution first, but after compiling my project to create the distributable **macOS** app, it stopped working - see info on it later.

```
{
  "name": "Launch Electron",
  "type": "node",
  "program": "${workspaceRoot}/start.js",
  "request": "launch",
  "stopOnEntry": false,
  "args": [],
  "cwd": "${workspaceRoot}",
  "runtimeExecutable": "${workspaceRoot}/node_modules/.bin/electron"
}
```

In the case of the **web app**, you can just add the **client id** and **client secret** to the **URL** as **query params**, so that you don't have to type them each time:
https://oss-manager.autodesk.io/?**client_id**=<client id>&**client_secret**=<client secret>

I needed a way to store this info for the user in the case of the **desktop app** as well. Based on this solution, I could just expose a couple of endpoints on the **server-side** to **store** and **retrieve** custom data from the **client**. I placed all that in a new file called **user.data.js**:

```
'use strict'; // http://www.w3schools.com/js/js_strict.asp
```

```
// web framework
var express = require('express');
var router = express.Router();

var bodyParser = require('body-parser');
var jsonParser = bodyParser.json();

const electron = require('electron');
const path = require('path');
const fs = require('fs');

const userDataFileName = 'buckets-tools.json';

router.get('/userdata', function (req, res) {
    let userDataPath = (electron.app || electron.remote.app).getPath('userData');
    try {
        userDataPath = path.join(userDataPath, userDataFileName);
        let json = JSON.parse(fs.readFileSync(userDataPath));
        res.json(json);
    } catch (ex) {
        res.status(404).end("No user data");
    }
})

router.post('/userdata', jsonParser, function (req, res) {
    try {
        let data = JSON.stringify(req.body);
        let userDataPath = (electron.app || electron.remote.app).getPath('userData');
        userDataPath = path.join(userDataPath, userDataFileName);
        fs.writeFileSync(userDataPath, data);
        res.end("Saved user data");
    } catch (ex) {
        res.status(500).end("Failed to save user data");
    }
})

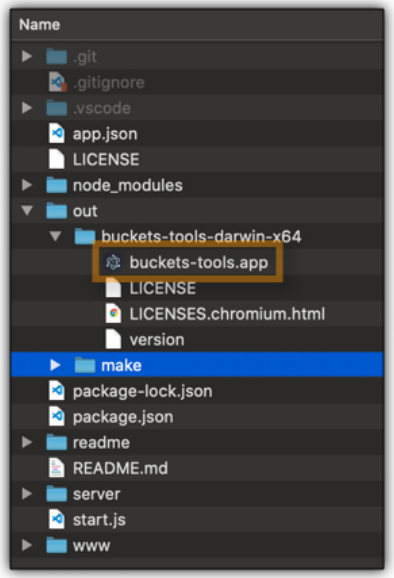
////////////////////////////////////////////////////////////////////
// Return the router object that contains the endpoints
////////////////////////////////////////////////////////////////////
module.exports = router;
```

Now I can call it from the **client-side** code to populate the **client id** and **client secret** edit boxes automatically:

```
$.ajax({
  url: "/userdata"
})
 done(function(data) {
```

```
  console.log(data);
  $("#client_id").val(data.clientId);
  $("#client_secret").val(data.clientSecret);
})
.fail(function(xhr, ajaxOptions, thrownError) {
  console.log("No user data");
});
```

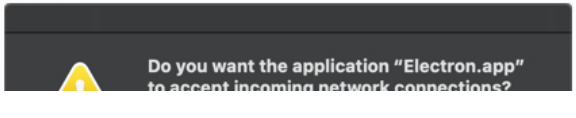
Then it was time to generate the **desktop app** based on the info found here. The result showed up in the **out** folder and the app worked fine

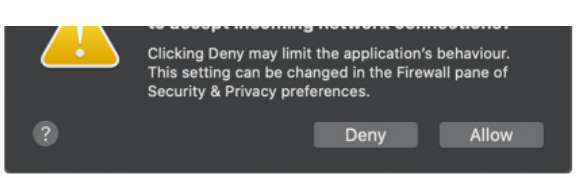


However, this last step broke my **launch.json** file that needed to be updated to this:

```
{
  "name": "Launch Electron New",
  "type": "node",
  "request": "launch",
  "stopOnEntry": false,
  "args": ["run", "start"],
  "cwd": "${workspaceRoot}",
  "runtimeExecutable": "npm"
}
```

The app worked fine (see image on top), but this **message box** kepts popping up every time I started the app





I don't have access to **Firewall** settings, and it would be effective only on my machine, so I was checking how else it could be avoided. It seemed that signing the app might do the trick, so I tried this:

```
(base) C02XJG9QJG5J:forge-buckets-tools nagyad$ codesign -s - -f ./out/buckets-tools-darwin-x6
xcrun: error: invalid active developer path (/Library/Developer/CommandLineTools), missing xcr
./out/buckets-tools-darwin-x64/buckets-tools.app: the codesign_allocate helper tool cannot be
In subcomponent: /Users/nagyad/Documents/GitHub/Autodesk-Forge/forge-buckets-tools/out/buckets
```

Dealing With Electron Error Message

As you can see I got an error because I did not have Xcode installed on my new machine - I had not needed it for ages. So I installed that and tried again but was told that I had not signed the license agreement using **sudo xcodebuild -license**
So I did that: I had to press **space** many times and type "**agree**" at the very end.

This time around **codesign -s - -f ./out/buckets-tools-darwin-x64/buckets-tools.app --deep** succeeded and when running the app I did not get the message box ?
I hope it works for other people as well, not just me.

Then I went to my **Windows 10** machine to generate the **Windows** executable but ran into some problems. I've done so many things, I'm not even sure if any of them really helped, apart from installing the latest version of **Python**.

First I got this error message when trying to run **npm run make** on **Windows**:

```
√ Checking your system
It looks like you are missing some dependencies you need to get Electron running.
Make sure you have git installed and Node.js version >= 10.0.0
```

Among many things, I also tried running **npm i** on my project and I got this:

```
npm ERR! code 1
npm ERR! path C:\Temp\forge-buckets-tools\node_modules\lzma-native
npm ERR! command failed
npm ERR! command C:\Windows\system32\cmd.exe /d /s /c node-gyp rebuild
npm ERR! gyp info it worked if it ends with ok
npm ERR! gyp info using node-gyp@7.1.2
npm ERR! gyp info using node@15.5.1 | win32 | x64
```

```
npm ERR! gyp ERR! find Python
```

So I installed the latest version of **Python** from https://www.python.org/downloads/ then **npm run make** worked if I started **Visual Studio Code** with the **Run as administrator** option:

```
> buckets-tools@2.0.0 make
> electron-forge make

√ Checking your system
√ Resolving Forge Config
We need to package your application before we can make it
Downloading electron-v11.2.2-win32-x64.zip: [============================================================
√ Preparing to Package Application for arch: x64
√ Preparing native dependencies
√ Packaging Application
Making for the following targets: squirrel
√ Making for target: squirrel - On platform: win32 - For arch: x64
```

Note: for all commands above I used the **Terminal** inside **Visual Studio Code** - in case it makes a difference.

Electron Buckets Tool Source Code Links

The source code is available in the **electron** branch of the **Buckets Tools** repo: https://github.com/Autodesk-Forge/forge-buckets-tools/tree/electron
The root folder also contains the **executables** for both **macOS** and **Windows**.

Download Mac Version: **HERE**

and window version: **HERE**

TAGS: Authentication, Data Management API, Model Derivative API, Viewer, Code Samples

POSTED BY



Adam Nagy

Follow @AdamTheNagy Adam Nagy joined Autodesk back in 2005 and has been providing programming support, consulting, training and evangelism to external

Resources

Get help on StackOverflow →

Explore code samples →



developers. He started his career in Budapest, then worked in Prague for 3 years and now lives in South England, UK. At the moment focusing on Inventor and Fusion 360, plus cloud and mobile related technologies. Adam has a degree in Software Engineering and has...

Find a systems integrator →

Contact Autodesk Platform Services →

© 2023 Autodesk Inc. All rights reserved.